Autonomous Intelligent Reinforcement Inferred Symbolism

Berick Cook[1] and Patrick Hammer[2]

[1] SingularityNET
[2] KTH Royal Institute of Technology

Abstract. This paper introduces AIRIS (Autonomous Intelligent Reinforcement Inferred Symbolism) to enable causality-based artificial intelligent agents. The system builds sets of causal rules from observations of changes in its environment which are typically caused by the actions of the agent. These rules are similar in format to rules in expert systems, however rather than being human-written, they are learned entirely by the agent itself as it keeps interacting with the environment.

Keywords: Procedure Learning · Experiential Learning · Autonomous Agent · Causal Reasoning · Artificial General Intelligence

1 Introduction

With expert systems [3] it is possible to encode simple sets of rules which can represent causal rules inherent in the environment or application domain the AI system is utilized in. These rules govern how the system (or agents of the system) perceive the environment, plan actions, and perform tasks. The rules are hand-coded by human programmers who dictate the behaviors of the system. In environments such as video games, these rules can result in intelligent agents that are capable of sophisticated interactions with their environment, other agents, and the player(s). Unfortunately, these rules are brittle and prone to failing when situations arise that are outside the rules provided by the programmer.

AIRIS is a method for autonomously learning expert system-like casual rules from raw observations of its operating environment. Each rule describes partial changes of state (rather than full state transitions) and together can be used to generate future predicted states via a voting mechanism that takes rule precondition mismatch into account. On a high level this leads to an action state graph which represents an internal world model. However, compared to state transition models as in Markov Decision Processes (MDP, [8]) the agent can construct future states which have not previously been experienced by the agent. The system then uses any planning algorithm on this world model to make plans and perform tasks, while giving it the flexibility to deal with non-stationary environments and changing objectives beyond what reinforcement learning agents [5] with the typical MDP formalization can achieve.

AIRIS retains all of the benefits of expert systems such as transparency, mutability, scrutability, and effectivity while also providing the flexibility and

adaptability that expert systems lack. AIRIS is continuously learning and not limited to its training. When it encounters situations that are outside the rules it has been trained for, it learns new rules that it can then apply to similar situations. Differently to most reasoning systems, it supports rule induction, which happens while the system is operating and interacting with the environment. Also the system does not need prior logical knowledge to be given, which also sets it apart from projects like Cyc [4]. Additionally, it does not demand logical encodings of inputs, such as as needed by Non-Axiomatic Reasoning System [10]) and planning systems using Planning Domain Definition Language (PDDL, [1]). Instead, it can directly build hypotheses describing action-dependent changes of observable values. AIRIS can be considered an Experiential Learning System which uses an empirical form of causality, which is based on correlation but involving the actions of the agent. This is fundamentally different to Pearl's approach [9] which needs a causal model (a causal graph) to be pre-specified, instead of the agent seeking for (and testing) potentially causal rules through interactions with the environment.

2 Reinforcement Inferred Causality

Expert system rules are comprised of a multitude of conditional statements. The format of these rules is typically IF $<$ *conditions* $>$ THEN $<$ *statements* $>$ or a form of logical implication statements [4]. The conditions of these rules are boolean evaluated to determine if the statement should be executed or leads to the described outcome.

AIRIS[3] uses similar conditional statements, referred to as Rules, that it generates through observations. These Rules are used to predict part of a future state by applying the Rules to the current State and storing the newly modified State in a State Graph. The State Graph represents the system's internal world model, and is used to make plans and achieve goals.

Rule Creation: When the agent performs an action, it compares the state of its inputs to the predicted State stored in the State Graph. For each value that is observed to change unexpectedly, a Rule is generated specifically for that value. Therefore the number of Rules that are generated when a change occurs is equivalent to the number of unexpected values that changed. The Conditions for a Rule consist of the action that was taken, the Original Value which is what the value was prior to the action taken, the Relative Change Positions which are the positions relative to the Original Value's position whose values also changed, and the Relative Change Original Values which are what the values in those relative positions were prior to the action taken. The Statement is what the Original Value changed into. When a new Rule is created, the agent's existing plan is abandoned, and the State Graph is cleared. A new State Graph is then generated that incorporates the new Rules.

[3] The current implementation is available in the corresponding open-source repository on Github: https://github.com/berickcook/AIRIS_Public

State Graph Creation: A set of Focus Values are compiled from the values in the current State that changed from the previous State. These Focus Values reduce the computational complexity by narrowing the agents attention from every value in the State to a small set of values. It evaluates every Rule for every Focus Value in the State. It computes a precondition match score to evaluate how similar each condition of a Rule is to the current state $(0 : 1)$. The precondition match score is calculated by comparing the Rule's Condition data with the values in the current State. These precondition match scores are stored in a priority queue. The highest scoring Rule has its Statement applied to the Focus Value. The average of all the highest scoring Rules is the State Confidence $(0 : 1)$ of the prediction. The values in the positions in the highest scoring Rule's Relative Change Positions are added to the Focus Values. This ensures that other relevant values in the State are evaluated if they were not part of the original Focus Values. The resulting modified State is stored in the State Graph with an Edge labeled with the action taken and State Confidence of the prediction connecting it back to the original State. This results in a self-generated State Graph that mirrors the style and usefulness of finite-state machines. And since it is self-generated the possible states are not pre-specified, they result from the application of the causal rules which capture changes of individual values rather than the complete state. This fosters the generalization abilities of AIRIS as it allows the system to use its knowledge in novel situations it has never seen before.

State Graph Usage: The State Graph initially consists of the current state of the environment inputs, and any number of unconnected Goal Sub-States. Goal Sub-States are generated sub-states that consist solely of the conditions of Rules where the Statement results in achieving the agent's specified Goal. If the agent has not yet observed a change that resulted in achieving the agent's Goal, then the State Graph only consists of the current environment State. If such Goal Sub-States exist, then as each new state is added to the State Graph it is heuristically evaluated to determine how close or far it is from all Goal Sub-States. This heuristic guides the Planning process to generate new States towards the closest unconnected Goal Sub-State. When a new state is generated that contains the Goal Sub-State, a Goal State has been found. The action path from the current State to the Goal State becomes the agent's plan. If no Goal Sub-States exist or the Goal Sub-States cannot be reached, then the State with the lowest Confidence becomes the Goal State. This results in the agent pursuing novel situations to observe and generate new Rules. The new Rules may allow it to create a Goal Sub-State or predict a new path to an existing Goal Sub-State. This can be considered as an effective model of artificial curiosity, which does not only capture aspects of information gain as in [7], but leads to deliberate interactions planned by the agent using the empirically-causal knowledge it possesses.

3 Architecture

The architecture of AIRIS consists of four primary components (Fig. 1). The Rule Base of learned rules, the State Graph that contains the world model of current and future predicted states, the Pre-Action Observation sequence, and the Post-Action Observation sequence.

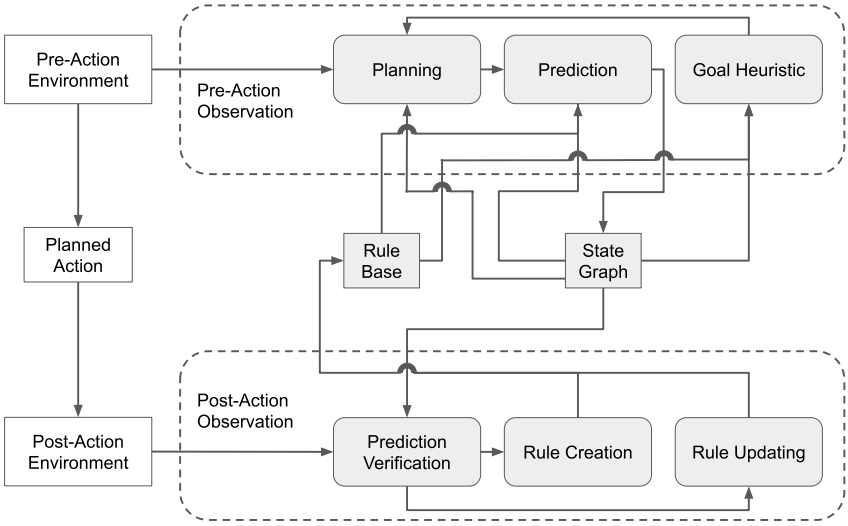

Fig. 1. AIRIS architecture diagram

Pre-Action Environment: The collection of sensory inputs from the operating environment prior to an action being taken.

It consist of sub-symbolic values which capture the presence of different object types at the existing locations of the environment.

Pre-Action Observation: This is the sequence of processes to make predictions of states to store in the State Graph and search across the State Graph for sequences of actions that result in achieving the current Goal.

Planning: This process guides the generation of new prediction States towards Goal Sub-States. If a Goal State is found, it traces the sequence of actions necessary to traverse the State Graph from the current State to the Goal State and sequentially feeds the actions back to the environment. If there are no Goal Sub-States or Goal Sub-States cannot be reached then the State with the lowest State Confidence becomes the Goal State.

Prediction: This process takes a given state from the Planning process and predicts what the next state will be for each action available based on the rules in the Rule Base. These predicted states are then added to the State Graph.

Goal Heuristic: This is a dynamically calculated heuristic value that is assigned to each state in the State Graph. It takes every rule from the Rule Base whose Statement results in achieving its current goal, and compares it to a given state from the State Graph. If the values in the conditions of the rule exist in the state data, it calculates the relative Manhattan distance between those values and assigns the distance as the heuristic value for that state. A State's heuristic value is used by the Planning process to guide predictions toward the Goal.

Post-Action Environment: The collection of sensory inputs from the operating environment after an action being taken.

Post-Action Observation: This is the sequence of processes to analyze observations of changes that occurred when the Planned Action was performed, and to create new rules or update the data in existing rules.

Prediction Verification: This process compares the Post-Action Environment data to the predicted State data to verify that a prediction was correct. If there is any discrepancy, then the action plan is abandoned, the State Graph is cleared, and new rules are created for the failed predictions. If there is no discrepancy, then the rules that the prediction was based on are updated with any relevant information.

Rule Creation: If the Prediction Verification process finds a discrepancy, then the Rule Creation process builds new rules that represent the unexpected changes. These rules are then added to the Rule Base.

Rule Updating: If the Prediction Verification process does not find a discrepancy, the Rule Updating process updates the rules that were applied in the accurate prediction.

4 Data Structures

There are two primary classes of data: Rules and States. Rules consist of any number of Conditions which were observed to be true when a value in the environment changed, and what the value changed into. Conditions consist of the relative position of other values in the environment that also changed, what those values changed into, and a list of other rules that were observed to be true for the prediction. States consist of copies of the environment data that have been modified by the Prediction process and represent future predicted states, a list of the Rules that were applied during the Prediction process, the action that was performed that resulted in the State, and a reference to the prior State.

5 Experiments & Comparisons

AIRIS has been tested in several different environments to examine its general purpose capabilities.

Grid World Puzzle Game: The primary testing environment has been a simple grid world puzzle game. The purpose of this environment is to test its ability to learn simple game rules, use those rules to solve problems and complete

tasks with causal reasoning, demonstrate the ability to change its objectives with no impact on efficacy, and demonstrate the overall transparency and scrutability.

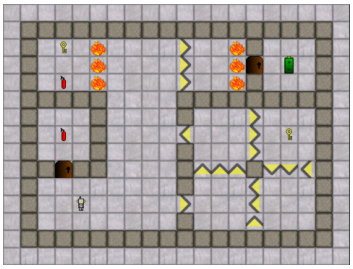

Fig. 2. Level 12 of the Grid World Puzzle Game

Inputs consist of a two dimensional array of integers that represent the objects in the game world, and a one dimensional array of integers that represent the inventory of items collected by the agent. Outputs consist of four actions: up, left, down, and right. The game consists of a series of thirteen levels of varying difficulty. The game objects consist of the agent, walls, batteries, one-way arrows, doors, keys, fire, and fire extinguishers. The agent completes a level by collecting all of the batteries. One-way arrows cannot be moved into from the direction they are pointing, but can be moved into from any of the other three directions. Doors require a key to open which consumes the key. Fire restarts the level on touch and subtracts one battery, but can be put out by a fire extinguisher on touch which consumes the fire extinguisher. There is no time limit.

The untrained agent began at level one and played sequentially through the levels in one continuous episode. If the agent touched fire and restarted a level, the episode did not end. It was only given the Goal of maximizing the number of batteries collected. The agent first discovered how to collect a battery after 32 timesteps. The agent was then able to complete all levels in the first episode in 4,612 timesteps. This demonstrates that the agent is able to adapt to novel situations such as encountering new obstacles and levels without the need for extensive training over multiple episodes.

The trained agent was placed back at level one and it was able to complete all levels in the second episode in 921 timesteps. This demonstrates that the agent is able to utilize the rules it learned to rapidly improve its efficacy.

The trained agent was placed into a new level that it had not been trained on that contained all game objects. It was able to solve it in 67 timesteps (Best possible is 54 timesteps). This demonstrates that the agent is not limited to levels it has been trained on, and can apply causal reasoning to achieve goals in novel level configurations.

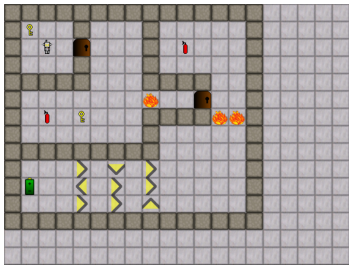

Fig. 3. AIRIS can complete a level it has never seen

The trained agent was placed in level eight. Before it could collect the final battery, its Goal was changed to decrease batteries collected. It immediately abandoned collecting the battery and went into the fire. When the level reset, it ignored the doors, keys, and batteries and went into the fire again. It continued to decrease its battery count until the Goal was changed back. This demonstrates that the Goal of the agent can be changed at any time.

The trained agent was placed back at level one with the Goal of decreasing its battery count. It is not possible for the agent to decrease its battery count until it encounters fire in level seven. The agent took slightly longer than usual to complete the levels, as it explored predictions with less than 100% confidence. It only collected batteries when there was nothing more for it to explore. When it reached level seven it immediately and repeatedly touched the fire. This demonstrates that even when its Goal is not possible the agent will explore until it can achieve its Goal even if the Goal was not what it was originally trained on.

The trained agent was placed back at level one and a visualization tool was used to view the agent's internal world model and see what its next plan is. When the visualization tool is active, the agent waits for a command to proceed before executing its plan. This demonstrates the scrutability of the system as are able to directly interface with the agent's internal world model to see what it has learned, what its predictions are, and what it is planning to do. The visualization tool can be expanded upon to show any additional details that the user may find relevant, or allow the user to control and "play" the internal world model as though the user was playing the game itself (as demonstrated by NACE).

Cartpole: This classic control problem which is concerned to balance a pole on a cart [2], tested the ability of AIRIS to work in a continuous space for potential future robotic applications, as well as tested it against a well established Reinforcement Learning benchmark.

Inputs consist of a one dimensional, floating-point array containing the Cart Position, Cart Velocity, Pole Angle, and Pole Angular Velocity. Outputs consist of two actions: push cart left and push cart right. The Goal is to keep the Pole Angle as close to 0 as possible for 200 time steps. If the Pole Angle exceeds $\pm 12^{\circ}$ or the Cart Position is greater than ± 2.4 then the episode ends. Achieving a

score of 195 over the last 100 episodes is considered solved. The environment also provides a reward, but it was not utilized as the goal condition is considered in planning instead.

AIRIS was able to reach its first score of 200 after 32 episodes and solved the environment after 130 episodes with consistent success. Comparatively, DQN took over 60 episodes to reach its first score of 200, and solved the environment after 200 episodes with varying success.

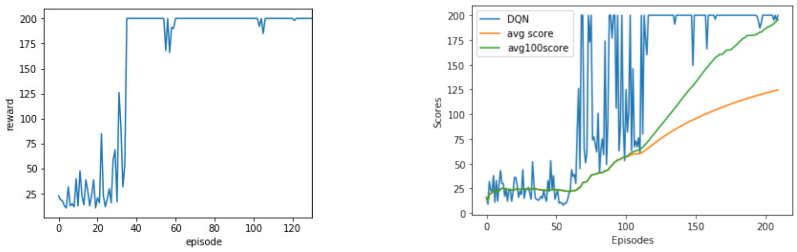

Fig. 4. AIRIS Cartpole results (left) and DQN Cartpole results (right)

MNIST: This classic image recognition problem is concerned about recognition of handwritten digits. With this dataset we tested the potential multi-modal capabilities of AIRIS. While Convolutional Neural Networks represent the state-of-the-art in this dataset in terms of accuracy after training, rule-based learning like in AIRIS can lead to data-efficient learning and hence also deserves to be investigated.

Inputs consist of a two dimensional array of pixel data from a "Training" set of 60,000 handwritten numbers and a "Test" set of another 10,000 numbers, and a single value one dimensional array of the label of the current image. Output consists of a single action "Reveal Label". An image is shown to the system with a null label. The system must then predict what value the label will be when the action is performed.

On the first run of the "Training" set, AIRIS's accuracy quickly rose and finished with 91% accuracy. The trained agent was then given the "Test" set where its accuracy dropped to a low of 90% before finishing at 93%. On the second run of the "Training" set, it had a consistent accuracy of 97%. On the second run of the "Test" set it had a consistent accuracy of 99%. Comparatively, the deep Convolutional Neural Network [6] took several hundred runs to reach equivalent accuracy.

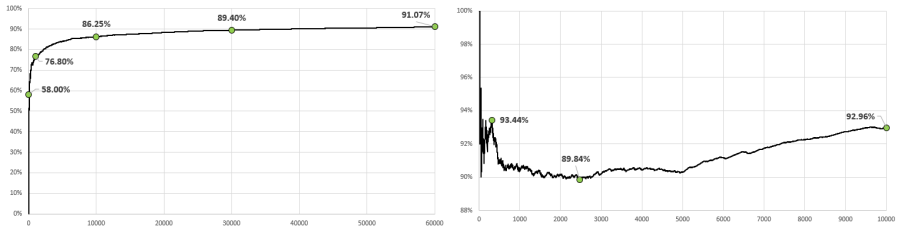

Fig. 5. AIRIS MNIST training set (left) and test set (right) results from first run

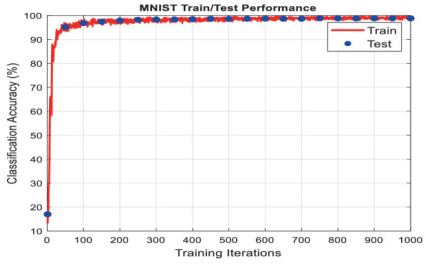

Fig. 6. Optimized Convolutional Neural Network Model Results

6 Limitations

As of this paper, AIRIS is not able to learn rules for events that occur regardless of the agent's action. Such as the independent movements of other agents or objects. This limits the system to environments where the only events that occur are caused directly by the actions of the agent. Environments where all input values can change, such as video feeds, pose a computational cost limitation. AIRIS will likely require additional attention mechanisms to prioritize processing of values relevant to the current task and disregarding other values to optimize the computational cost. Additionally, while an extension to three dimensions is straightforward, more research is needed for continuous spaces or high-resolution semantic grid maps. The latter will be required for letting AIRIS operate autonomous mobile robots, as grid maps provide a suitable representation to make AIRIS well-integrated with Robot Operating System. We expect that this will allow the system to operate in real-world scenarios when variants of Semantic Simultaneous Localization and Mapping are utilized.

7 Conclusion

Despite its current limitations, AIRIS is a promising alternative to contemporary Reinforcement Learning techniques. In grid world environments it can learn from a fraction of the training that RL techniques require, with substantially better efficacy due to its causal representations, and has the ability to immediately switch to different objectives. Its ability to continuously learn allows it to adapt to new situations outside of its training, and its transparency and scrutability offers many advantages over RL. This system can help to increase the generality of autonomous agents and the data-efficiency of learning goal-directed behaviors.

Future work will address the aforementioned limitations: extending the model to deal with agent-external changes, partial observability, and non-determism, as well as to extend the model to work in continuous (or high-resolution) three-dimensional environments. Also, we will more systematically analyze the inductive biases which make AIRIS so data-efficient.

References

1. Aeronautiques, C., Howe, A., Knoblock, C., McDermott, I.D., Ram, A., Veloso, M., Weld, D., Sri, D.W., Barrett, A., Christianson, D., et al.: Pddl— the planning domain definition language. Technical Report, Tech. Rep. (1998)
2. Geva, S., Sitte, J.: A cartpole experiment benchmark for trainable controllers. IEEE Control Systems Magazine **13**(5), 40–51 (1993)
3. Giarratano, J.C., Riley, G.: Expert systems. PWS Publishing Co. (1998)
4. Lenat, D.B., Guha, R.V., Pittman, K., Pratt, D., Shepherd, M.: Cyc: toward programs with common sense. Communications of the ACM **33**(8), 30–49 (1990)
5. Moerland, T.M., Broekens, J., Plaat, A., Jonker, C.M., et al.: Model-based reinforcement learning: A survey. Foundations and Trends® in Machine Learning **16**(1), 1–118 (2023)
6. Salman, O.S., Salman, A.S.: Addressing challenging problems using optimized deep learning classification algorithms on the mnist dataset. In: Future of Information and Communication Conference. pp. 247–260. Springer (2022)
7. Still, S., Precup, D.: An information-theoretic approach to curiosity-driven reinforcement learning. Theory in Biosciences **131**, 139–148 (2012)
8. Sutton, R.S., Barto, A.G.: Reinforcement learning: An introduction. MIT press (2018)
9. Verma, T., Pearl, J.: Causal networks: Semantics and expressiveness. In: Machine intelligence and pattern recognition, vol. 9, pp. 69–76. Elsevier (1990)
10. Wang, P.: Non-axiomatic logic: A model of intelligent reasoning. World Scientific (2013)